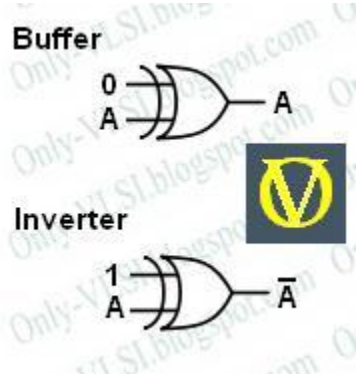
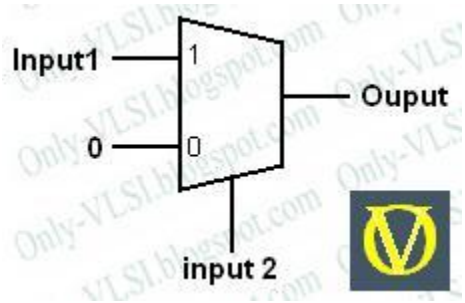
http://only-vlsi.blogspot.in/2009/01/digital-design-interview-questions.html

Digital Design Interview Questions - All in 1

1. How do you convert a XOR gate into a buffer and a inverter (Use only one XOR gate for each)?
Answer



2. Implement an 2-input AND gate using a 2x1 mux.
Answer



3. What is a multiplexer?
Answer

A multiplexer is a combinational circuit which selects one of many input signals and directs to the only output.

4. What is a ring counter?
Answer

A ring counter is a type of counter composed of a circular shift register. The output of the last shift register is fed to the input of the first register. For example, in a 4-register counter, with initial register values of 1100, the repeating pattern is: 1100, 0110, 0011, 1001, 1100, so on.

5. Compare and Contrast Synchronous and Asynchronous reset.
Answer

Synchronous reset logic will synthesize to smaller flip-flops, particularly if the reset is gated with the logic generating the d-input. But in such a case, the combinational logic gate count grows, so the overall gate count savings may not be that significant. The clock works as a filter for small reset glitches; however, if these glitches occur near the active clock edge, the Flip-flop could go metastable. In some designs, the reset must be generated by a set of internal conditions. A synchronous reset is recommended for these types of designs because it will filter the logic equation glitches between clock.
Problem with synchronous resets is that the synthesis tool cannot easily distinguish the reset signal from any other data signal. Synchronous resets may need a pulse stretcher to guarantee a reset pulse width wide enough to ensure reset is present during an active edge of the clock, if you have a gated clock to save power, the clock may be disabled coincident with the assertion of reset. Only an asynchronous reset will work in this situation, as the reset might be removed prior to the resumption of the clock. Designs that are pushing the limit for data path timing, can not afford to have added gates and additional net delays in the data path due to logic inserted to handle synchronous resets.

Asynchronous reset: The major problem with asynchronous resets is the reset release, also called reset removal. Using an asynchronous reset, the designer is guaranteed not to have the reset added to the data path. Another advantage favoring asynchronous resets is that the circuit can be reset with or without a clock present. Ensure that the release of the reset can occur within one clock period else if the release of the reset occurred on or near a clock edge then flip-flops may go into metastable state.

6. What is a Johnson counter?
Answer

Johnson counter connects the complement of the output of the last shift register to its input and circulates a stream of ones followed by zeros around the ring. For example, in a 4-register counter, the repeating pattern is: 0000, 1000, 1100, 1110, 1111, 0111, 0011, 0001, so on.

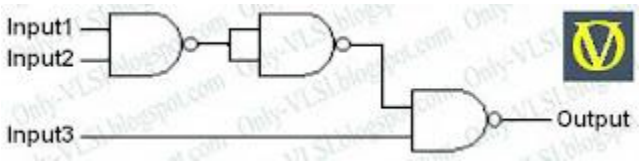
7. An assembly line has 3 fail safe sensors and one emergency shutdown switch.The line should keep moving unless any of the following conditions arise:
(1) If the emergency switch is pressed
(2) If the senor1 and sensor2 are activated at the same time.
(3) If sensor 2 and sensor3 are activated at the same time.
(4) If all the sensors are activated at the same time
Suppose a combinational circuit for above case is to be implemented only with NAND Gates. How many minimum number of 2 input NAND gates are required?
Answer

Solve it out!

8. In a 4-bit Johnson counter How many unused states are present?
Answer

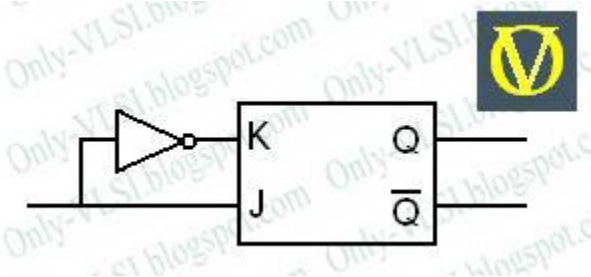
4-bit Johnson counter: 0000, 1000, 1100, 1110, 1111, 0111, 0011, 0001, 0000.
8 unused states are present.

9. Design a 3 input NAND gate using minimum number of 2 input NAND gates.
Answer



10. How can you convert a JK flip-flop to a D flip-flop?
Answer

Connect the inverted J input to K input.



11. What are the differences between a flip-flop and a latch?
Answer

Flip-flops are edge-sensitive devices where as latches are level sensitive devices.
Flip-flops are immune to glitches where are latches are sensitive to glitches.
Latches require less number of gates (and hence less power) than flip-flops.
Latches are faster than flip-flops.

12. What is the difference between Mealy and Moore FSM?
Answer

Mealy FSM uses only input actions, i.e. output depends on input and state. The use of a Mealy FSM leads often to a reduction of the number of states.
Moore FSM uses only entry actions, i.e. output depends only on the state. The advantage of the Moore model is a simplification of the behavior.

13. What are various types of state encoding techniques? Explain them.
Answer

One-Hot encoding: Each state is represented by a bit flip-flop). If there are four states then it requires four bits (four flip-flops) to represent the current state. The valid state values are 1000, 0100, 0010, and 0001. If the value is 0100, then it means second state is the current state.

One-Cold encoding: Same as one-hot encoding except that '0' is the valid value. If there are four states then it requires four bits (four flip-flops) to represent the current state. The valid state values are 0111, 1011, 1101, and 1110.

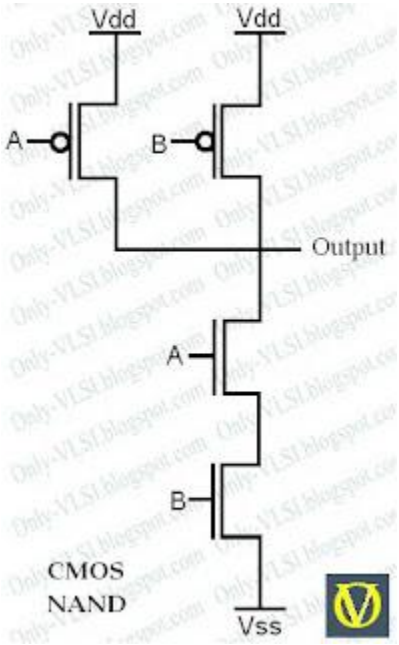
Binary encoding: Each state is represented by a binary code. A FSM having '2 power N' states requires only N flip-flops.

Gray encoding: Each state is represented by a Gray code. A FSM having '2 power N' states requires only N flip-flops.

14. Define Clock Skew , Negative Clock Skew, Positive Clock Skew.
Answer

Clock skew is a phenomenon in synchronous circuits in which the clock signal (sent from the clock circuit) arrives at different components at different times. This can be caused by many different things, such as wire-interconnect length, temperature variations, variation in intermediate devices, capacitive coupling, material imperfections, and differences in input capacitance on the clock inputs of devices using the clock.
There are two types of clock skew: negative skew and positive skew. Positive skew occurs when the clock reaches the receiving register later than it reaches the register sending data to the receiving register. Negative skew is the opposite: the receiving register gets the clock earlier than the sending register.

15. Give the transistor level circuit of a CMOS NAND gate.
Answer



16. Design a 4-bit comparator circuit.
Answer

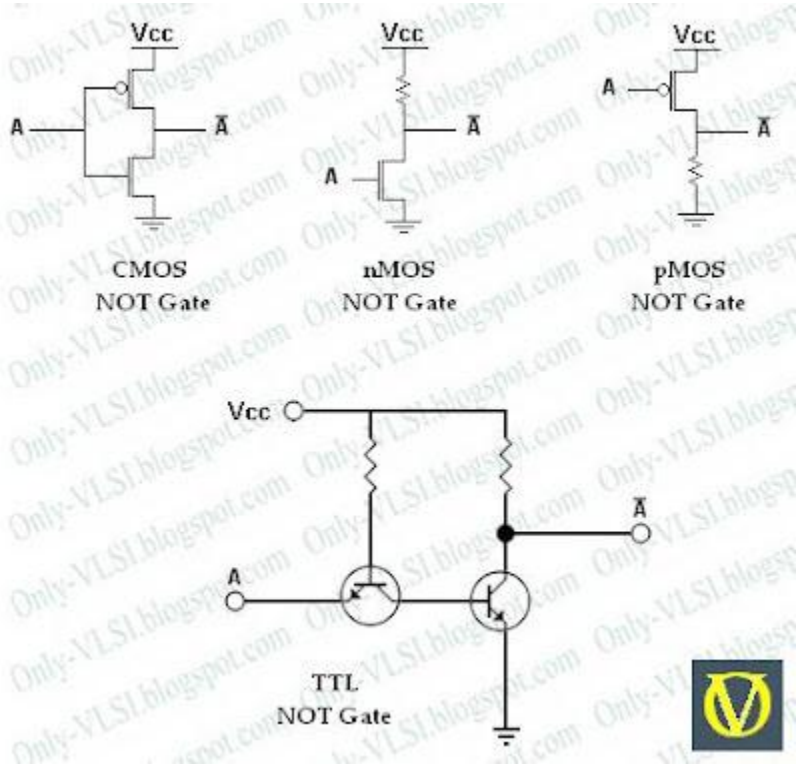
17. Design a Transmission Gate based XOR. Now, how do you convert it to XNOR (without inverting the output)?
Answer

18. Define Metastability.
Answer

If there are setup and hold time violations in any sequential circuit, it enters a state where its output is unpredictable, this state is known as metastable state or quasi stable state, at the end of metastable state, the flip-flop settles down to either logic high or logic low. This whole process is known as metastability.

19. Compare and contrast between 1's complement and 2's complement notation.
Answer

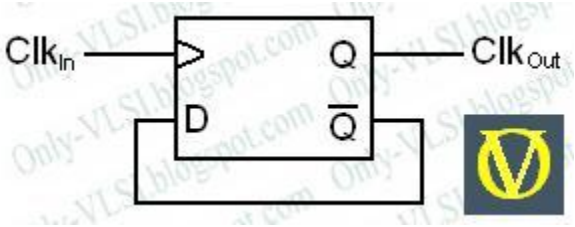
20. Give the transistor level circuit of CMOS, nMOS, pMOS, and TTL inverter gate.
Answer



21. What are set up time and hold time constraints?
Answer

Set up time is the amount of time before the clock edge that the input signal needs to be stable to guarantee it is accepted properly on the clock edge.
Hold time is the amount of time after the clock edge that same input signal has to be held before changing it to make sure it is sensed properly at the clock edge.
Whenever there are setup and hold time violations in any flip-flop, it enters a state where its output is unpredictable, which is known as as metastable state or quasi stable state. At the end of metastable state, the flip-flop settles down to either logic high or logic low. This whole process is known as metastability.

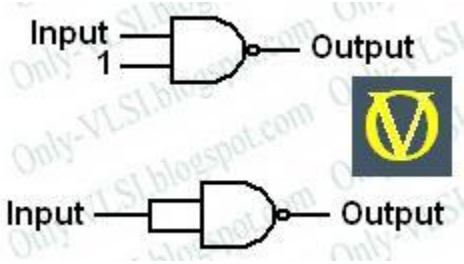
22. Give a circuit to divide frequency of clock cycle by two.
Answer



23. Design a divide-by-3 sequential circuit with 50% duty circle.
Answer

24. Explain different types of adder circuits.
Answer

25. Give two ways of converting a two input NAND gate to an inverter.
Answer



26. Draw a Transmission Gate-based D-Latch.
Answer

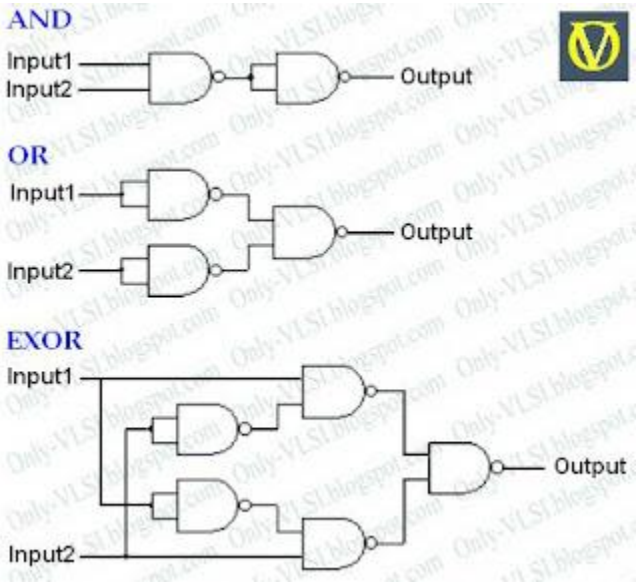
27. Design a FSM which detects the sequence 10101 from a serial line without overlapping.
Answer

28. Design a FSM which detects the sequence 10101 from a serial line with overlapping.
Answer

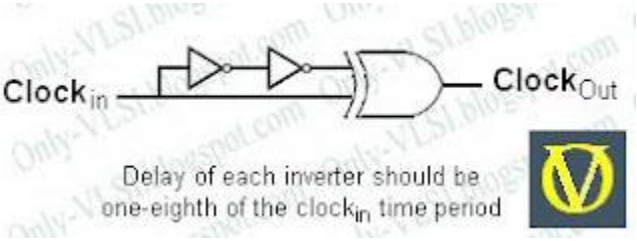
29. Give the design of 8x1 multiplexer using 2x1 multiplexers.
Answer

30. Design a counter which counts from 1 to 10 (Resets to 1, after 10).
Answer

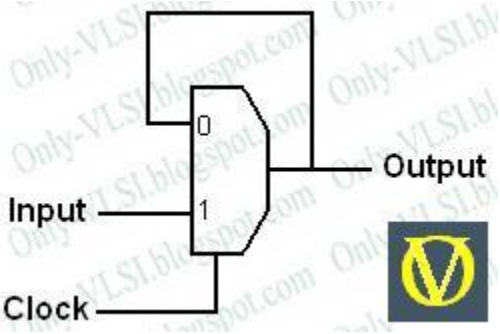
31. Design 2 input AND, OR, and EXOR gates using 2 input NAND gate.
Answer



32. Design a circuit which doubles the frequency of a given input clock signal.
Answer



33. Implement a D-latch using 2x1 multiplexer(s).
Answer



34. Give the excitation table of a JK flip-flop.
Answer

35. Give the Binary, Hexadecimal, BCD, and Excess-3 code for decimal 14.
Answer

14:
Binary: 1110
Hexadecimal: E
BCD: 0001 0100
Excess-3: 10001

36. What is race condition?
Answer

37. Give 1's and 2's complement of 19.
Answer

19: 10011
1's complement: 01100
2's complement: 01101

38. Design a 3:6 decoder.
Answer

39. If A*B=C and C*A=B then, what is the Boolean operator * ?
Answer

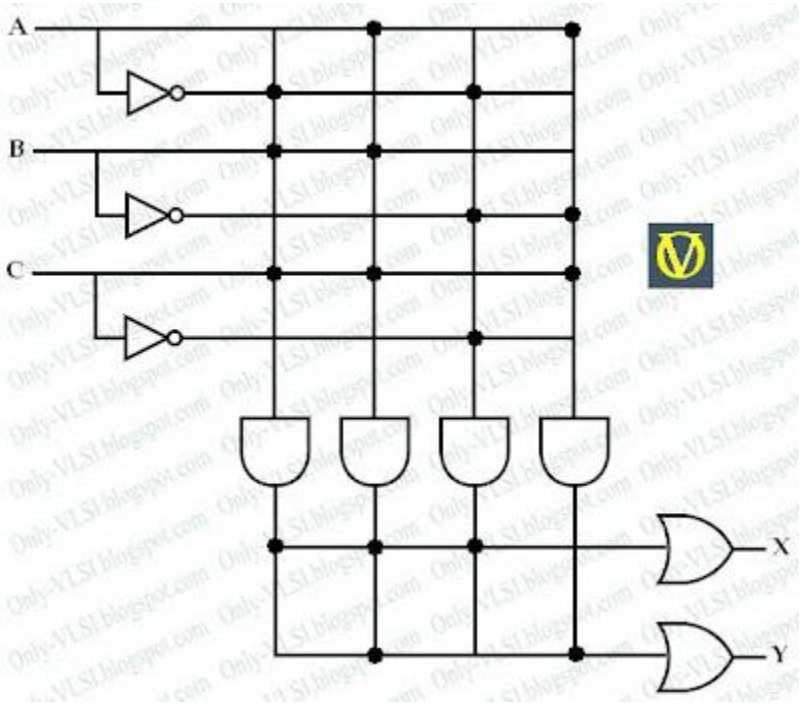
* is Exclusive-OR.

40. Design a 3 bit Gray Counter.
Answer

41. Expand the following: PLA, PAL, CPLD, FPGA.
Answer

PLA - Programmable Logic Array
PAL - Programmable Array Logic
CPLD - Complex Programmable Logic Device
FPGA - Field-Programmable Gate Array

42. Implement the functions: X = A'BC + ABC + A'B'C' and Y = ABC + AB'C using a PLA.
Answer



43. What are PLA and PAL? Give the differences between them.
Answer

Programmable Logic Array is a programmable device used to implement combinational logic circuits. The PLA has a set of programmable AND planes, which link to a set of programmable OR planes, which can then be conditionally complemented to produce an output.
PAL is programmable array logic, like PLA, it also has a wide, programmable AND plane. Unlike a PLA, the OR plane is fixed, limiting the number of terms that can be ORed together. Due to fixed OR plane PAL allows extra space, which is used for other basic logic devices, such as multiplexers, exclusive-ORs, and latches. Most importantly, clocked elements, typically flip-flops, could be included in PALs. PALs are also extremely fast.

44. What is LUT?
Answer

LUT - Look-Up Table. An n-bit look-up table can be implemented with a multiplexer whose select lines are the inputs of the LUT and whose inputs are constants. An n-bit LUT can encode any n-input Boolean function by modeling such functions as truth tables. This is an efficient way of encoding Boolean logic functions, and LUTs with 4-6 bits of input are in fact the key component of modern FPGAs.

45. What is the significance of FPGAs in modern day electronics? (Applications of FPGA.)
Answer

- ASIC prototyping: Due to high cost of ASIC chips, the logic of the application is first verified by dumping HDL code in a FPGA. This helps for faster and cheaper testing. Once the logic is verified then they are made into ASICs.
- Very useful in applications that can make use of the massive parallelism offered by their architecture. Example: code breaking, in particular brute-force attack, of cryptographic algorithms.
- FPGAs are sued for computational kernels such as FFT or Convolution instead of a microprocessor.
- Applications include digital signal processing, software-defined radio, aerospace and defense systems, medical imaging, computer vision, speech recognition, cryptography, bio-informatics, computer hardware emulation and a growing range of other areas.

46. What are the differences between CPLD and FPGA.
Answer

47. Compare and contrast FPGA and ASIC digital designing.
Answer

Click here.

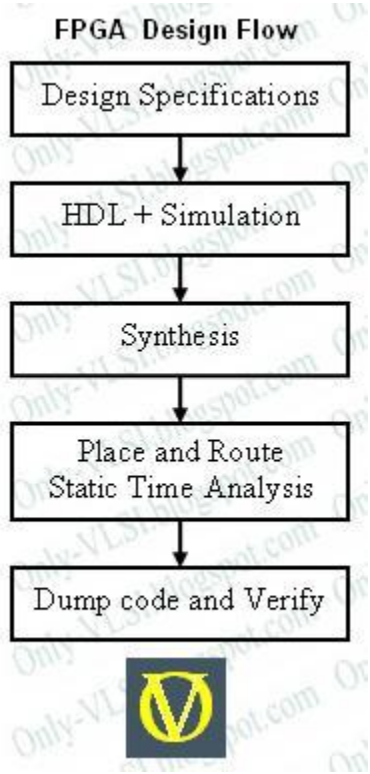
48. Give True or False.
(a) CPLD consumes less power per gate when compared to FPGA.
(b) CPLD has more complexity than FPGA
(c) FPGA design is slower than corresponding ASIC design.
(d) FPGA can be used to verify the design before making a ASIC.
(e) PALs have programmable OR plane.
(f) FPGA designs are cheaper than corresponding ASIC, irrespective of design complexity.
Answer

(a) False
(b) False
(c) True
(d) True
(e) False
(f) False

49. Arrange the following in the increasing order of their complexity: FPGA,PLA,CPLD,PAL.
Answer

Increasing order of complexity: PLA, PAL, CPLD, FPGA.

50. Give the FPGA digital design cycle.
Answer



51. What is DeMorgan's theorem?
Answer

For N variables, DeMorgan's theorems are expressed in the following formulas:
(ABC..N)' = A' + B' + C' + ... + N' -- The complement of the product is equivalent to the sum of the complements.
(A + B + C + ... + N)' = A'B'C'...N' -- The complement of the sum is equivalent to the product of the complements.
This relationship so induced is called DeMorgan's duality.

52. F'(A, B, C, D) = C'D + ABC' + ABCD + D. Express F in Product of Sum form.
Answer

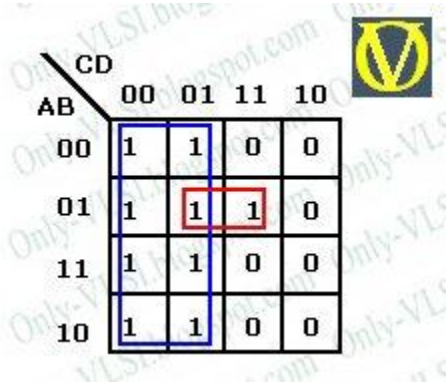
Complementing both sides and applying DeMorgan's Theorem:
F(A, B, C, D) = (C + D')(A' + B' + C)(A' + B' + C' + D')(D')

53. How many squares/cells will be present in the k-map of F(A, B, C)?
Answer

F(A, B, C) has three variables/inputs.
Therefore, number of squares/cells in k-map of F = $2^{(\text{Number of variables})} = 2^3 = 8$.

54. Simplify F(A, B, C, D) = S (0, 1, 4, 5, 7, 8, 9, 12, 13)
Answer

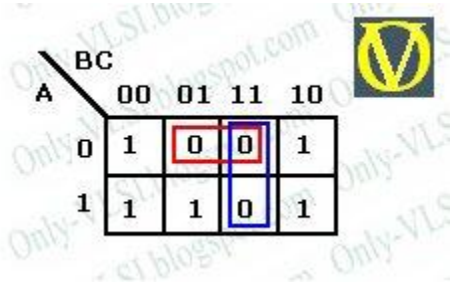
The four variable k-map of the given expression is:



The grouping is also shown in the diagram. Hence we get,
F(A, B, C, D) = C' + A'BD

55. Simplify F(A, B, C) = S (0, 2, 4, 5, 6) into Product of Sums.
Answer

The three variable k-map of the given expression is:



The 0's are grouped to get the F'.
F' = A'C + BC

Complementing both sides and using DeMorgan's theorem we get F,
F = (A + C')(B' + C')

56. The simplified expression obtained by using k-map method is unique. True or False. Explain your answer.
Answer

False. The simplest form obtained is not necessarily unique as grouping can be made in different ways.

57. Give the characteristic tables of RS, JK, D and T flip-flops.
Answer

RS flip-flop.

S R Q(t+1)
0 0 Q(t)
0 1 0
1 0 1
1 1 ?

JK flip-flop
J K Q(t+1)
0 0 Q(t)
0 1 0
1 0 1
1 1 Q'(t)

D flip-flop
D Q(t+1)
0 0
1 1

T flip-flop
T Q(t+1)
0 Q(t)
1 Q'(t)

58. Give excitation tables of RS, JK, D and T flip-flops.
Answer

RS flip-flop.

Q(t)	Q(t+1)	S	R
0	0	0	X
0	1	1	0
1	0	0	1
1	1	X	0

JK flip-flop

Q(t)	Q(t+1)	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

D flip-flop

Q(t)	Q(t+1)	D
0	0	0
0	1	1
1	0	0
1	1	1

T flip-flop

Q(t)	Q(t+1)	T
0	0	0
0	1	1
1	0	1
1	1	0

59. Design a BCD counter with JK flip-flops
Answer

60. Design a counter with the following binary sequence 0, 1, 9, 3, 2, 8, 4 and repeat. Use T flip-flops.
Answer

Microprocessor Interview Questions - 5

1. Why are program counter and stack pointer 16-bit registers?
Answer

Program Counter (PC) and Stack Pointer (SP) are basically used to hold 16-bit memory addresses.PC stores the 16-bit memory address of the next instruction to be fetched. SP stores address of stack's starting block.

2. What happens during DMA transfer?
Answer

During DMA transfers DMA controller takes control of the data transfer, and the processor will carry out other tasks.

3. Define ISR.
Answer

An interrupt handler, also known as an interrupt service routine (ISR), is a callback subroutine in an operating system or device driver whose execution is triggered by the reception of an interrupt. Whenever there is an interrupt the processor jumps to ISR and executes it.

4. Define PSW.
Answer

The Program Status Word (PSW) is a register which contains information about the current program status used by the operating system and the underlying hardware. The PSW includes the instruction address, condition code, and other fields. In general, the PSW is used to control instruction sequencing and to hold and indicate the status of the system in relation to the program currently being executed. The active or controlling PSW is called the current PSW. By storing the current PSW during an interruption, the status of the CPU can be preserved for subsequent inspection. By loading a new PSW or part of a PSW, the state of the CPU can be initialized or changed.

5. What are the execution modes available in x86 processors?
Answer

* Real mode (16-bit)
* Protected mode (16-bit and 32-bit)
* Virtual 8086 mode (16-bit)
* Unreal mode (32-bit)
* System Management Mode (16-bit)
* Long mode (64-bit)

6. What is meant real mode?
Answer

Real mode is an execution/operating mode of 80286 and later x86-compatible CPUs. Real mode is characterized by a 20 bit segmented memory address space, where a maximum of 1 MB of memory can be addressed, direct software access to BIOS routines and peripheral hardware, and no concept of memory protection or multitasking at the hardware level. All x86 CPUs in the 80286 series and later start in real mode at power-on (earlier CPUs had only one operational mode, which is equivalent to real mode in later chips).

7. What is protected mode?
Answer

Protected mode allows system software to utilize features such as virtual memory, paging, safe multi-tasking, and other features designed to increase an operating system's control over application software.
When a processor that supports x86 protected mode is powered on, it begins executing instructions in real mode, in order to maintain backwards compatibility with earlier x86

processors. Protected mode may only be entered after the system software sets up several descriptor tables and enables the Protection Enable (PE) bit in the Control Register 0.

8. What is virtual 8086 mode?
Answer

Virtual real mode or VM86, allows the execution of real mode applications that are incapable of running directly in protected mode. It uses a segmentation scheme identical to that of real mode, and also uses 21-bit addressing - resulting in linear addressing - so it is subject to paging.

9. What is unreal mode?
Answer

Unreal mode, also known as big real mode, huge real mode, or flat real mode, is a variant of real mode. one or more data segment registers will be loaded with 32-bit addresses and limits.

10. What is the difference between ISR and a function call?
Answer

ISR has no return value, where as a function call has the return value.

Microprocessor Interview Questions - 4

1. What is the size of flag register of 8086 processor?
Answer

16-bit.

2. How many pin IC 8086 is?
Answer

40 pin dual in-line package.

3. What is the Maximum clock frequency of 8086?
Answer

5 Mhz is the Maximum clock frequency of 8086.

4. What is meant by instruction cycle?
Answer

An instruction cycle also known as fetch-and-execute cycle and fetch-decode-execute cycle, is the time period during which a computer reads and processes a machine language instruction from its memory.

5. What is Von Neumann architecture?
Answer

The Von Neumann architecture is a computer design model that uses a processing unit and a single separate storage structure to hold both instructions and data. The instruction/data is read from storage and executed by the processing unit. It is also known as "stored-program computer".

6. What is the main difference between 8086 and 8085?
Answer

8086 is 16-bit microprocessor, where as 8085 is a 8-bit microprocessor.

7. What does EAX mean?
Answer

With the advent of the 32-bit 80386 processor, the 16-bit general-purpose registers, base registers, index registers, instruction pointer, and FLAGS register, but not the segment registers, were expanded to 32 bits. This is represented by prefixing an "E" (for Extended) to the register opcodes, thus the expanded AX became EAX.(Similarly BX became EBX, SI became ESI and so on).

8. What type of instructions are available in instruction set of 8086?
Answer

- Data Instructions
- Arithmetic Instructions
- Logic Instructions
- Control Instructions
- Other - setting/clearing flag bits, stack operations, software interrupts, etc.

9. How is Stack Pointer affected when a PUSH and POP operations are performed?
Answer

When PUSH operation is performed the SP value is decreased by 2. When POP operation is performed the SP value is increased by 2.

10. What are SIM and RIM instructions?
Answer

SIM - Set Interrupt Mask, is used to mask the hardware interrupts. RIM - Read Interrupt Mask, is used to check whether the interrupt is Masked or not.

Microprocessor Interview Questions - 3

1. How many bits processor is 8086?
Answer

16-bit processor.

2. What are the sizes of data bus and address bus in 8086?
Answer

16-bit data bus, and 20-bit address bus.

3. What is the maximum addressable memory of 8086?
Answer

1MByte, because 20-bit address bus.

4. How are 32-bit addresses stored in 8086?
Answer

32-bit addresses are stored in "SEGMENT:OFFSET" format. SEGMENT and OFFSET are 16-bit values.
ADDRESS = (SEGMENT* 16) + OFFSET

5. What are the 16-bit registers that are available in 8086?
Answer

The following are the 16-bit registers that are available in 8086.

8 general purpose registers:
AX - Accumulator Register
Bx - Base Register
CX - Count Register
DX - Data Register
SP - Stack Pointer
BP - Base Pointer
SI - Source Index
DI - Destination Index

4 segment registers:
CS - Code Segment
DS - Data Segment
SS - Stack Segment
ES - Extra Segment

Others:
IP - Instruction Pointer
Flag register

6. What are the different types of address modes available in 8086?
Answer

Implied - the data value/data address is implicitly associated with the instruction.
Register - references the data in a register or in a register pair.
Immediate - the data is provided in the instruction.
Direct - the instruction operand specifies the memory address where data is located.
Register indirect - instruction specifies a register containing an address, where data is located. This addressing mode works with SI, DI, BX and BP registers.
Based - 8-bit or 16-bit instruction operand is added to the contents of a base register (BX or BP), the resulting value is a pointer to location where data resides.
Indexed - 8-bit or 16-bit instruction operand is added to the contents of an index register (SI or DI), the resulting value is a pointer to location where data resides.
Based Indexed - the contents of a base register (BX or BP) is added to the contents of an index register (SI or DI), the resulting value is a pointer to location where data resides.
Based Indexed with Offset - 8-bit or 16-bit instruction operand is added to the contents of a base register (BX or BP) and index register (SI or DI), the resulting value is a pointer to location where data resides.

7. How many flags are available in flag register? What are they?
Answer

9 flags are available, they are:
Overflow Flag
Direction Flag
Interrupt-enable Flag
Trace/Trap Flag
Sign Flag
Zero Flag
Auxiliary carry Flag
Parity Flag
Carry Flag

8. Explain the functioning of IP (instruction pointer).
Answer

IP always points to next instruction to be executed. Offset address is relative to CS (which points at the segment containing the current program). The next instruction address is obtained using IP.

9. What are the various types of interrupts present in 8086?
Answer

INTR - maskable hardware interrupt
NMI - non-maskable interrupt
Software interrupts

10. How many segments are present in 8086? What are they?
Answer

4 segments are available in 8086. They are:
Code segment
Data segment
Extra segment
Stack segment

SoC : System-On-a-Chip

System-on-a-chip (SoC) refers to integrating all components of an electronic system into a single integrated circuit (chip). A SoC can include the integration of:

- Ready made sub-circuits (IP)
- One or more microcontroller, microprocessor or DSP core(s)
- Memory components
- Sensors
- Digital, Analog, or Mixed signal components
- Timing sources, like oscillators and phase-locked loops
- Voltage regulators and power management circuits

The blocks of SoC are connected by a special bus, such as the AMBA bus. DMA controllers are used for routing the data directly between external interfaces and memory, by-passing the processor core and thereby increasing the data throughput of the SoC. SoC is widely used in the area of embedded systems. SoCs can be fabricated by several technologies, like, Full custom, Standard cell, FPGA, etc. SoC designs are usually power and cost effective, and more reliable than the corresponding multi-chip systems. A programmable SoC is known as PSoC.

Advantages of SoC are:

- Small size, reduction in chip count
- Low power consumption
- Higher reliability
- Lower memory requirements
- Greater design freedom
- Cost effective

Design Flow

SoC consists of both hardware and software(to control SoC components). The aim of SoC design is to develop hardware and software in parallel. SoC design uses pre-qualified hardware, along with their software (drivers) which control them. The hardware blocks are put together using CAD tools; the software modules are integrated using a software development environment. The SoC design is then programmed onto a FPGA, which helps in testing the behavior of SoC. Once SoC design passes the testing it is then sent to the place and route process. Then it will be fabricated. The chips will be completely tested and verified.

0 Comments

Labels: Integrated Circuits

Complex Programmable Logic Device

A complex programmable logic device (CPLD) is a semiconductor device containing programmable blocks called macro cell, which contains logic implementing disjunctive normal form expressions and more specialized logic operations. CPLD has complexity between that of PALs and FPGAs. It can has up to about 10,000 gates. CPLDs offer very predictable timing characteristics and are therefore ideal for critical control applications.

Applications

- CPLDs are ideal for critical, high-performance control applications.
- CPLD can be used for digital designs which perform boot loader functions.
- CPLD is used to load configuration data for an FPGA from non-volatile memory.
- CPLD are generally used for small designs, for example, they are used in simple applications such as address decoding.
- CPLDs are often used in cost-sensitive, battery-operated portable applications, because of its small size and low-power usage.

Architecture

A CPLD contains a bunch of programmable functional blocks (FB) whose inputs and outputs are connected together by a global interconnection matrix. The global interconnection matrix is reconfigurable, so that we can change the connections between the FBs. There will be some I/O blocks which allow us to connect CPLD to external world. The block diagram of architecture of CPLD is shown below.

The programmable functional block typically looks like the one shown below. There will be an array of AND gates which can be programed. The OR gates are fixed. But each manufacturer has their way of building the functional block. A registered output can be obtained by manipulating the feedback signals obtained from the OR ouputs.

CPLD Programming

The design is first coded in HDL (Verilog or VHDL), once the code is validated (simulated and synthesized). During synthesis the target device(CPLD model) is selected, and a technology-mapped net list is generated. The net list can then be fitted to the actual CPLD architecture using a process called place-and-route, usually performed by the CPLD company's proprietary place-and-route software. Then the user will do some verification processes. If every thing is fine, he will use the CPLD, else he will reconfigure it.

1 Comments

Labels: Integrated Circuits

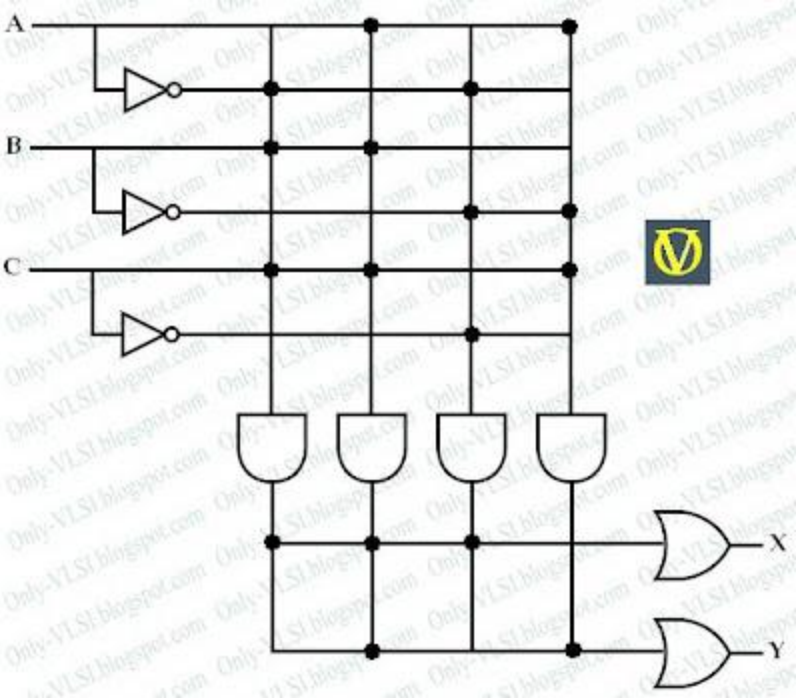
Programmable Logic Array

In Digital design, we often use a device to perform multiple applications. The device configuration is changed (reconfigured) by programming it. Such devices are known as programmable devices. It is used to build reconfigurable digital circuits. The following are the popular programmable device

- PLA - Programmable Logic Array
- PAL - Programmable Array Logic
- CPLD - Complex Programmable Logic Device (Click here for more details)
- FPGA - Field-Programmable Gate Array (Click here for more details)

PLA: Programmable Logic Array is a programmable device used to implement combinational logic circuits. The PLA has a set of programmable AND planes, which link to a set of programmable OR planes, which can then be conditionally complemented to produce an output. This layout allows for a large number of logic functions to be synthesized in the sum of products canonical forms.

Suppose we need to implement the functions: $X = A'BC + ABC + A'B'C'$ and $Y = ABC + AB'C$. The following figures shows how PLA is configured. The big dots in the diagram are connections. For the first AND gate (left most), *A* complement, *B*, and *C* are connected, which is first minterm of function *X*. For second AND gate (from left), *A*, *B*, and *C* are connected, which forms *ABC*. Similarly for $A'B'C'$, and $AB'C$. Once the minterms are implemented. Now we have to combine them using OR gates to the functions *X*, and *Y*.



One application of a PLA is to implement the control over a data path. It defines various states in an instruction set, and produces the next state (by conditional branching).

Note that the use of the word "Programmable" does not indicate that all PLAs are field-programmable; in fact many are mask-programmed during manufacture in the same manner as a ROM. This is particularly true of PLAs that are embedded in more complex and numerous integrated circuits such as microprocessors. PLAs that can be programmed after manufacture are called FPLA (Field-programmable logic array).

0 Comments

Labels: Integrated Circuits

FPGA vs ASIC

Definitions

FPGA: A Field-Programmable Gate Array (FPGA) is a semiconductor device containing programmable logic components called "logic blocks", and programmable interconnects. Logic blocks can be programmed to perform the function of basic logic gates such as AND, and XOR, or more complex combinational functions such as decoders or mathematical functions. *For complete details click here.*

ASIC: An application-specific integrated circuit (ASIC) is an integrated circuit designed for a particular use, rather than intended for general-purpose use. Processors, RAM, ROM, etc are examples of ASICs.

FPGA vs ASIC

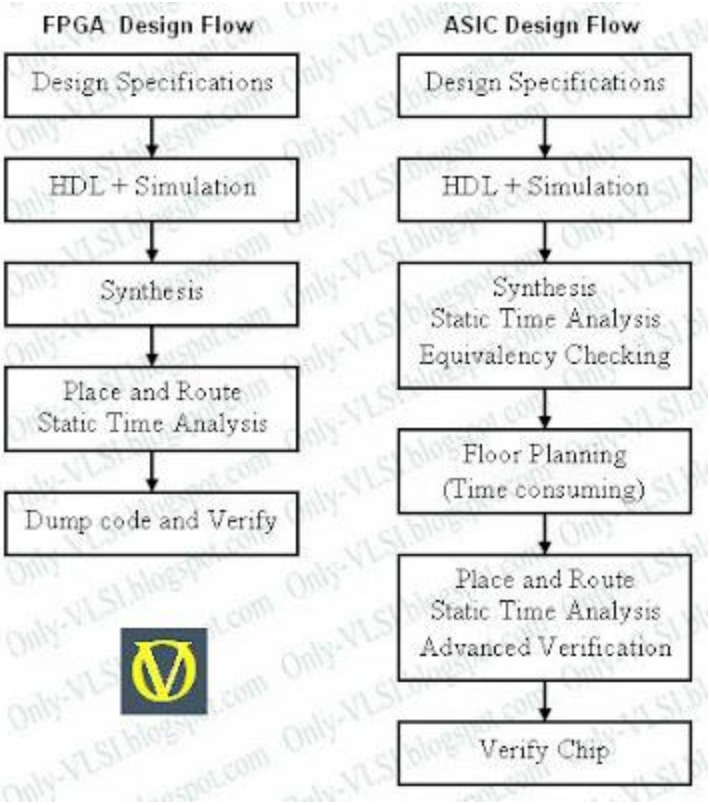
Speed
ASIC rules out FPGA in terms of speed. As ASIC are designed for a specific application they can be optimized to maximum, hence we can have high speed in ASIC designs. ASIC can have hight speed clocks.

Cost
FPGAs are cost effective for small applications. But when it comes to complex and large volume designs (like 32-bit processors) ASIC products are cheaper.

Size/Area
FPGA are contains lots of LUTs, and routing channels which are connected via bit streams(program). As they are made for general purpose and because of re-usability. They are in-general larger designs than corresponding ASIC design. For example, LUT gives you both registered and non-register output, but if we require only non-registered output, then its a waste of having a extra circuitry. In this way ASIC will be smaller in size.

Power
FPGA designs consume more power than ASIC designs. As explained above the unwanted circuitry results wastage of power. FPGA wont allow us to have better power optimization. When it comes to ASIC designs we can optimize them to the fullest.

Time to Market
FPGA designs will till less time, as the design cycle is small when compared to that of ASIC designs. No need of layouts, masks or other back-end processes. Its very simple: Specifications -- HDL + simulations -- Synthesis -- Place and Route (along with static-analysis) -- Dump code onto FPGA and Verify. When it comes to ASIC we have to do floor planning and also advanced verification. The FPGA design flow eliminates the complex and time-consuming floor planning, place and route, timing analysis, and mask / re-spin stages of the project since the design logic is already synthesized to be placed onto an already verified, characterized FPGA device.



Type of Design
ASIC can have mixed-signal designs, or only analog designs. But it is not possible to design them using FPGA chips.

Customization
ASIC has the upper hand when comes to the customization. The device can be fully customized as ASICs will be designed according to a given specification. Just imagine implementing a 32-bit processor on a FPGA!

Prototyping
Because of re-usability of FPGAs, they are used as ASIC prototypes. ASIC design HDL code is first dumped onto a FPGA and tested for accurate results. Once the design is error free then it is taken for further steps. Its clear that FPGA may be needed for designing an ASIC.

Non Recurring Engineering/Expenses
NRE refers to the one-time cost of researching, designing, and testing a new product, which is generally associated with ASICs. No such thing is associated with FPGA. Hence FPGA designs are cost effective.

Simpler Design Cycle
Due to software that handles much of the routing, placement, and timing, FPGA designs have smaller designed cycle than ASICs.

More Predictable Project Cycle
Due to elimination of potential re-spins, wafer capacities, etc. FPGA designs have better project

cycle.

Tools
Tools which are used for FPGA designs are relatively cheaper than ASIC designs.

Re-Usability
A single FPGA can be used for various applications, by simply reprogramming it (dumping new HDL code). By definition ASIC are application specific cannot be reused.

1 Comments

Labels: ASIC, FPGA, Integrated Circuits

Field-Programmable Gate Array

A *Field-Programmable Gate Array* (FPGA) is a semiconductor device containing programmable logic components called "logic blocks", and programmable interconnects. Logic blocks can be programmed to perform the function of basic logic gates such as AND, and XOR, or more complex combinational functions such as decoders or mathematical functions. In most FPGAs, the logic blocks also include memory elements, which may be simple flip-flops or more complete blocks of memory.

Applications

- ASIC prototyping: Due to high cost of ASIC chips, the logic of the application is first verified by dumping HDL code in a FPGA. This helps for faster and cheaper testing. Once the logic is verified then they are made into ASICs.
- Very useful in applications that can make use of the massive parallelism offered by their architecture. Example: code breaking, in particular brute-force attack, of cryptographic algorithms.
- FPGAs are sued for computational kernels such as FFT or Convolution instead of a microprocessor.
- Applications include digital signal processing, software-defined radio, aerospace and defense systems, medical imaging, computer vision, speech recognition, cryptography, bio-informatics, computer hardware emulation and a growing range of other areas.

Architecture

FPGA consists of large number of "configurable logic blocks" (CLBs) and routing channels. Multiple I/O pads may fit into the height of one row or the width of one column in the array. In general all the routing channels have the same width. The block diagram of FPGA architecture is shown below.

CLB: The CLB consists of an n-bit look-up table (LUT), a flip-flop and a 2x1 mux. The value n is manufacturer specific. Increase in n value can increase the performance of a FPGA. Typically n is 4. An n-

bit lookup table can be implemented with a multiplexer whose select lines are the inputs of the LUT and whose inputs are constants. An n-bit LUT can encode any n-input Boolean function by modeling such functions as truth tables. This is an efficient way of encoding Boolean logic functions, and LUTs with 4-6 bits of input are in fact the key component of modern FPGAs. The block diagram of a CLB is shown below.

Each CLB has n-inputs and only one input, which can be either the registered or the unregistered LUT output. The output is selected using a 2x1 mux. The LUT output is registered using the flip-flop (generally D flip-flop). The clock is given to the flip-flop, using which the output is registered. In general, high fanout signals like clock signals are routed via special-purpose dedicated routing networks, they and other signals are managed separately.

Routing channels are programmed to connect various CLBs. The connecting done according to the design. The CLBs are connected in such a way that logic of the design is achieved.

FPGA Programming

The design is first coded in HDL (Verilog or VHDL), once the code is validated (simulated and synthesized). During synthesis, typically done using tools like Xilinx ISE, FPGA Advantage, etc, a technology-mapped net list is generated. The net list can then be fitted to the actual FPGA architecture using a process called place-and-route, usually performed by the FPGA company's proprietary place-and-route software. The user will validate the map, place and route results via timing analysis, simulation, and other verification methodologies. Once the design and validation process is complete, the binary file generated is used to (re)configure the FPGA. Once the FPGA is (re)configured, it is tested. If there are any issues or modifications, the original HDL code will be modified and then entire process is repeated, and FPGA is reconfigured.

FPGA vs ASIC

Definitions

FPGA: A Field-Programmable Gate Array (FPGA) is a semiconductor device containing programmable logic components called "logic blocks", and programmable interconnects. Logic blocks can be programmed to perform the function of basic logic gates such as AND, and XOR, or more complex combinational functions such as decoders or mathematical functions. *For complete details click here.*

ASIC: An application-specific integrated circuit (ASIC) is an integrated circuit designed for a particular use, rather than intended for general-purpose use. Processors, RAM, ROM, etc are

examples of ASICs.

FPGA vs ASIC

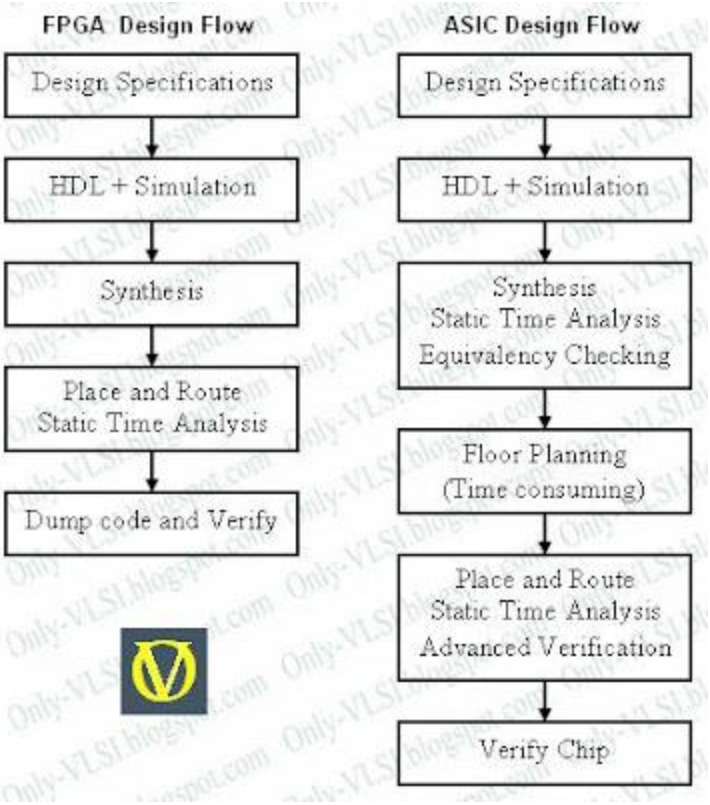
Speed
ASIC rules out FPGA in terms of speed. As ASIC are designed for a specific application they can be optimized to maximum, hence we can have high speed in ASIC designs. ASIC can have hight speed clocks.

Cost
FPGAs are cost effective for small applications. But when it comes to complex and large volume designs (like 32-bit processors) ASIC products are cheaper.

Size/Area
FPGA are contains lots of LUTs, and routing channels which are connected via bit streams(program). As they are made for general purpose and because of re-usability. They are in-general larger designs than corresponding ASIC design. For example, LUT gives you both registered and non-register output, but if we require only non-registered output, then its a waste of having a extra circuitry. In this way ASIC will be smaller in size.

Power
FPGA designs consume more power than ASIC designs. As explained above the unwanted circuitry results wastage of power. FPGA wont allow us to have better power optimization. When it comes to ASIC designs we can optimize them to the fullest.

Time to Market
FPGA designs will till less time, as the design cycle is small when compared to that of ASIC designs. No need of layouts, masks or other back-end processes. Its very simple: Specifications -- HDL + simulations -- Synthesis -- Place and Route (along with static-analysis) -- Dump code onto FPGA and Verify. When it comes to ASIC we have to do floor planning and also advanced verification. The FPGA design flow eliminates the complex and time-consuming floor planning, place and route, timing analysis, and mask / re-spin stages of the project since the design logic is already synthesized to be placed onto an already verified, characterized FPGA device.



Type of Design
ASIC can have mixed-signal designs, or only analog designs. But it is not possible to design them using FPGA chips.

Customization
ASIC has the upper hand when comes to the customization. The device can be fully customized as ASICs will be designed according to a given specification. Just imagine implementing a 32-bit processor on a FPGA!

Prototyping
Because of re-usability of FPGAs, they are used as ASIC prototypes. ASIC design HDL code is first dumped onto a FPGA and tested for accurate results. Once the design is error free then it is taken for further steps. Its clear that FPGA may be needed for designing an ASIC.

Non Recurring Engineering/Expenses
NRE refers to the one-time cost of researching, designing, and testing a new product, which is generally associated with ASICs. No such thing is associated with FPGA. Hence FPGA designs are cost effective.

Simpler Design Cycle
Due to software that handles much of the routing, placement, and timing, FPGA designs have smaller designed cycle than ASICs.

More Predictable Project Cycle
Due to elimination of potential re-spins, wafer capacities, etc. FPGA designs have better project

cycle.

Tools
Tools which are used for FPGA designs are relatively cheaper than ASIC designs.

Re-Usability
A single FPGA can be used for various applications, by simply reprogramming it (dumping new HDL code). By definition ASIC are application specific cannot be reused.

Synchronous Reset VS Asynchronous Reset

Why Reset?

A Reset is required to initialize a hardware design for system operation and to force an ASIC into a known state for simulation.

A reset simply changes the state of the device/design/ASIC to a user/designer defined state. There are two types of reset, what are they? As you can guess them, they are Synchronous reset and Asynchronous reset.

Synchronous Reset

A synchronous reset signal will only affect or reset the state of the flip-flop on the active edge of the clock. The reset signal is applied as is any other input to the state machine.

Advantages:

- The advantage to this type of topology is that the reset presented to all functional flip-flops is fully synchronous to the clock and will always meet the reset recovery time.
- Synchronous reset logic will synthesize to smaller flip-flops, particularly if the reset is gated with the logic generating the d-input. But in such a case, the combinational logic gate count grows, so the overall gate count savings may not be that significant.
- Synchronous resets provide some filtering for the reset signal such that it is not effected by glitches, unless they occur right at the clock edge. A synchronous reset is recommended for some types of designs where the reset is generated by a set of internal conditions. As the clock will filter the logic equation glitches between clock edges.

Disadvantages:

- The problem in this topology is with reset assertion. If the reset signal is not long enough to be captured at active clock edge (or the clock may be slow to capture the reset signal), it will result in failure of assertion. In such case the design needs a pulse stretcher to guarantee that a reset pulse is wide enough to be present during the active clock edge.
- Another problem with synchronous resets is that the logic synthesis cannot easily distinguish the reset signal from any other data signal. So proper care has to be taken with

logic synthesis, else the reset signal may take the fastest path to the flip-flop input there by making worst case timing hard to meet.
- In some power saving designs the clocked is gated. In such designed only asynchronous reset will work.
- Faster designs that are demanding low data path timing, can not afford to have extra gates and additional net delays in the data path due to logic inserted to handle synchronous resets.

Asynchronous Reset

An asynchronous reset will affect or reset the state of the flip-flop asynchronously i.e. no matter what the clock signal is. This is considered as high priority signal and system reset happens as soon as the reset assertion is detected.

Advantages:

- High speeds can be achieved, as the data path is independent of reset signal.
- Another advantage favoring asynchronous resets is that the circuit can be reset with or without a clock present.
- As in synchronous reset, no work around is required for logic synthesis.

Disadvantages:

- The problem with this type of reset occurs at logic de-assertion rather than at assertion like in synchronous circuits. If the asynchronous reset is released (reset release or reset removal) at or near the active clock edge of a flip-flop, the output of the flip-flop could go metastable.
- Spurious resets can happen due to reset signal glitches.

Conclusion

Both types of resets have positives and negatives and none of them assure fail-proof design. So there is something called "Asynchronous assertion and Synchronous de-assertion" reset which can be used for best results. (which will be discussed in next post).

9 Comments

Labels: ASIC, Digital Design, Important Concepts, VLSI design

One-hot Encoding

Designing a FSM is the most common and challenging task for every digital logic designer. One of the key factors for optimizing a FSM design is the choice of state coding, which influences the complexity of the logic functions, the hardware costs of the circuits, timing issues, power usage, etc. There are several options like binary encoding, gray encoding, one-hot encoding, etc. The choice of the designer depends on the factors like technology, design specifications, etc.

One-hot encoding

In one-hot encoding only one bit of the state vector is asserted for any given state. All other state bits are zero. Thus if there are *n* states then *n* state flip-flops are required. As only one bit remains logic high and rest are logic low, it is called as One-hot encoding.
Example: If there is a FSM, which has 5 states. Then 5 flip-flops are required to implement the FSM using one-hot encoding. The states will have the following values:
S0 - 10000
S1 - 01000
S2 - 00100
S3 - 00010
S4 - 00001

Advantages

- State decoding is simplified, since the state bits themselves can be used directly to check whether the FSM is in a particular state or not. Hence additional logic is not required for decoding, this is extremely advantageous when implementing a big FSM.
- Low switching activity, hence resulting low power consumption, and less prone to glitches.
- Modifying a design is easier. Adding or deleting a state and changing state transition equations (combinational logic present in FSM) can be done without affecting the rest of the design.
- Faster than other encoding techniques. Speed is independent of number of states, and depends only on the number of transitions into a particular state.
- Finding the critical path of the design is easier (static timing analysis).
- One-hot encoding is particularly advantageous for FPGA implementations. If a big FSM design is implemented using FPGA, regular encoding like binary, gray, etc will use fewer flops for the state vector than one-hot encoding, but additional logic blocks will be required to encode and decode the state. But in FPGA each logic block contains one or more flip-flops (click here to know why?) hence due to presence of encoding and decoding more logics block will be used by regular encoding FSM than one-hot encoding FSM.

Disadvantages

- The only disadvantage of using one-hot encoding is that it required more flip-flops than the other techniques like binary, gray, etc. The number of flip-flops required grows linearly with number of states. *Example*: If there is a FSM with 38 states. One-hot encoding requires 38 flip-flops where as other require 6 flip-flops only.

0 Comments

Labels: FSM, Important Concepts

Random Access Memory

Random Access Memory (RAM) is a type of computer data storage. Its mainly used as main memory of a computer. RAM allows to access the data in any order, i.e random. The word random thus refers to the fact that any piece of data can be returned in a constant time, regardless of its physical location and whether or not it is related to the previous piece of data. You can access any memory cell directly if you know the row and column that intersect at that cell.

Most of the RAM chips are volatile types of memory, where the information is lost after the power is switched off. There are some non-volatile types such as, ROM, NOR-Flash.

SRAM: Static Random Access Memory

SRAM is static, which doesn't need to be periodically refreshed, as SRAM uses bistable latching circuitry to store each bit. SRAM is volatile memory. Each bit in an SRAM is stored on four transistors that form two cross-coupled inverters. This storage cell has two stable states which are used to denote 0 and 1. Two additional access transistors serve to control the access to a storage cell during read and write operations. A typical SRAM uses six MOSFETs to store each memory bit.

As SRAM doesnt need to be refreshed, it is faster than other types, but as each cell uses at least 6 transistors it is also very expensive. So in general SRAM is used for faster access memory units of a CPU.

DRAM: Dynamic Random Access Memory

In a DRAM, a transistor and a capacitor are paired to create a memory cell, which represents a single bit of data. The capacitor holds the bit of information. The transistor acts as a switch that lets the control circuitry on the memory chip read the capacitor or change its state. As capacitors leak charge, the information eventually fades unless the capacitor charge is refreshed periodically. Because of this refresh process, it is a dynamic memory.

The advantage of DRAM is its structure simplicity. As it requires only one transistor and one capacitor per one bit, high density can be achieved. Hence DRAM is cheaper and slower, when compared to SRAM.

Other types of RAM

FPM DRAM: Fast page mode dynamic random access memory was the original form of DRAM. It waits through the entire process of locating a bit of data by column and row and then reading the bit before it starts on the next bit.

EDO DRAM: Extended data-out dynamic random access memory does not wait for all of the processing of the first bit before continuing to the next one. As soon as the address of the first bit is located, EDO DRAM begins looking for the next bit. It is about five percent faster than FPM.

SDRAM: Synchronous dynamic random access memory takes advantage of the burst mode concept to greatly improve performance. It does this by staying on the row containing the requested bit and moving rapidly through the columns, reading each bit as it goes. The idea is that most of the time the data needed by the CPU will be in sequence. SDRAM is about five percent faster than EDO RAM and is the most common form in desktops today.

DDR SDRAM: Double data rate synchronous dynamic RAM is just like SDRAM except that is has higher bandwidth, meaning greater speed.

DDR2 SDRAM: Double data rate two synchronous dynamic RAM. Its primary benefit is the ability to operate the external data bus twice as fast as DDR SDRAM. This is achieved by improved bus signaling, and by operating the memory cells at half the clock rate (one quarter of the data transfer rate), rather than at the clock rate as in the original DDR SRAM.

0 Comments

Labels: Important Concepts

Direct Memory Access

Direct memory access (DMA) is a feature of modern computers that allows certain hardware subsystems within the computer to access system memory for reading and/or writing independently of the central processing unit. Computers that have DMA channels can transfer data to and from devices with much less CPU overhead than computers without a DMA channel.

Principle of DMA

DMA is an essential feature of all modern computers, as it allows devices to transfer data without subjecting the CPU to a heavy overhead. Otherwise, the CPU would have to copy each piece of data from the source to the destination. This is typically slower than copying normal blocks of memory since access to I/O devices over a peripheral bus is generally slower than normal system RAM. During this time the CPU would be unavailable for any other tasks involving CPU bus access, although it could continue doing any work which did not require bus access.

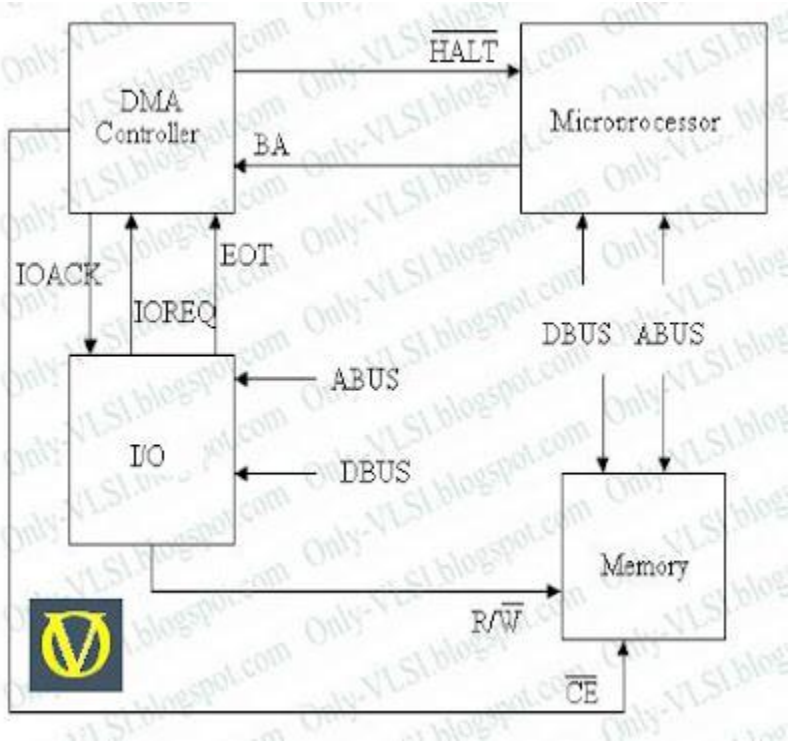
A DMA transfer essentially copies a block of memory from one device to another. While the CPU initiates the transfer, it does not execute it. For so-called "third party" DMA, as is normally used with the ISA bus, the transfer is performed by a DMA controller which is typically part of the motherboard chipset. More advanced bus designs such as PCI typically use bus mastering DMA, where the device takes control of the bus and performs the transfer itself.

A typical usage of DMA is copying a block of memory from system RAM to or from a buffer on the device. Such an operation does not stall the processor, which as a result can be scheduled to perform other tasks. DMA is essential to high performance embedded systems. It is also essential in providing so-called zero-copy implementations of peripheral device drivers as well as functionalities such as network packet routing, audio playback and streaming video.

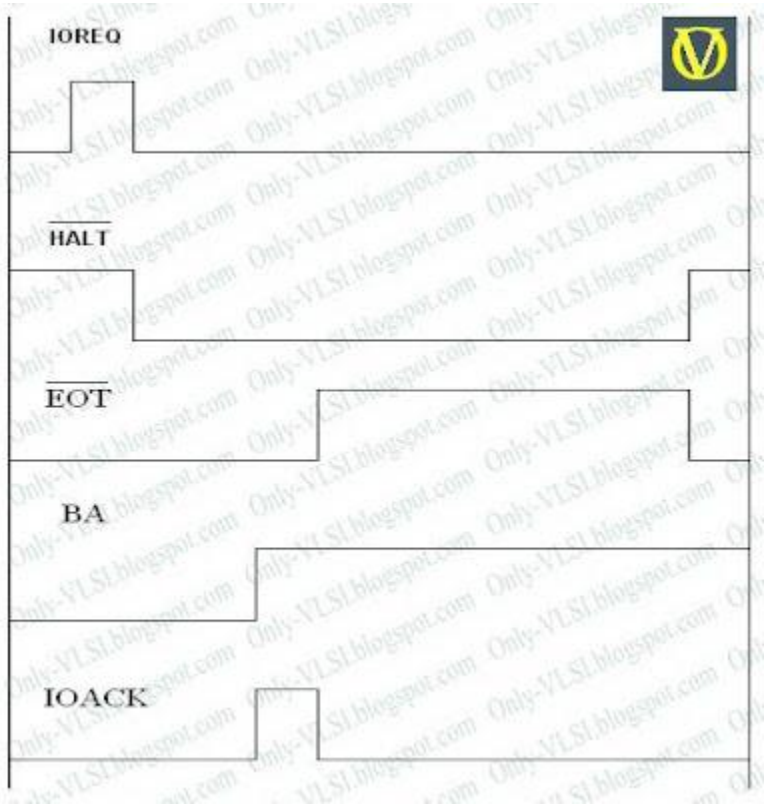
DMA Controller

The processing unit which controls the DMA process is known as DMA controller. Typically the job of the DMA controller is to setup a connection between the memory unit and the IO device,

with the permission from the microprocessor, so that the data can be transferred with much less processor overhead. The following figure shows a simple example of hardware interface of a DMA controller in a microprocessor based system.



Functioning (Follow the timing diagram for better understanding).
Whenever there is a IO request (IOREQ) for memory access from a IO device. The DMA controller sends a Halt signal to microprocessor. Generally halt signal (HALT) is active low. Microprocessor then acknowledges the DMA controller with a bus availability signal (BA). As soon as BA is available, then DMA controller sends an IO acknowledgment to IO device (IOACK) and chip enable (CE - active low) to the memory unit. The read/write control (R/W) signal will be give by the IO device to memory unit. Then the data transfer will begin. When the data transfer is finished, the IO device sends an end of transfer (EOT - active low) signal. Then the DMA controller will stop halting the microprocessor. ABUS and DBUS are address bus and data bus, respectively, they are included just for general information that microprocessor, IO devices, and memory units are connected to the buses, through which data will be transferred.



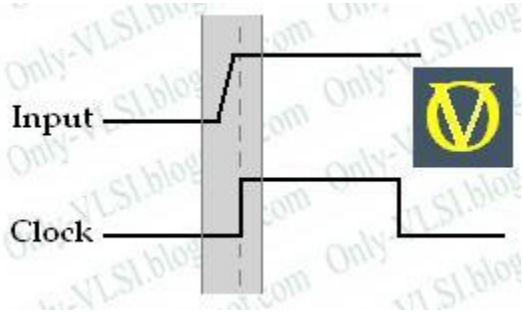
0 Comments

Labels: Important Concepts

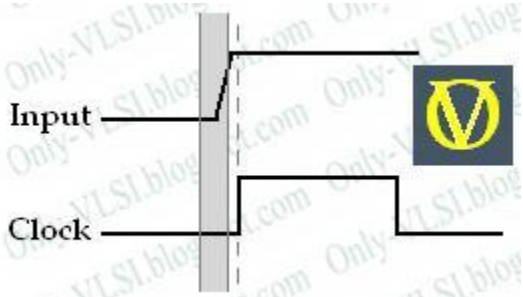
Setup and Hold TIme

Every flip-flop has restrictive time regions around the active clock edge in which input should not change. We call them restrictive because any change in the input in this regions the output may be the expected one (*see below). It may be derived from either the old input, the new input, or even in between the two. Here we define, two very important terms in the digital clocking. Setup and Hold time.

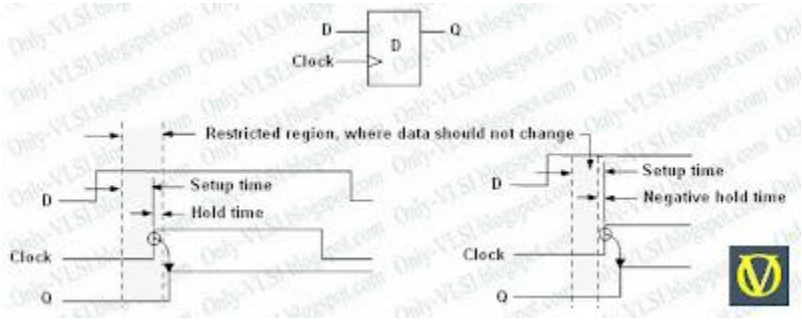
- The **setup time** is the interval before the clock where the data must be held stable.
- The **hold time** is the interval after the clock where the data must be held stable. Hold time can be negative, which means the data can change slightly before the clock edge and still be properly captured. Most of the current day flip-flops has zero or negative hold time.



In the above figure, the shaded region is the restricted region. The shaded region is divided into two parts by the dashed line. The left hand side part of shaded region is the setup time period and the right hand side part is the hold time period. If the data changes in this region, as shown the figure. The output may, follow the input, or many not follow the input, or may go to metastable state (where output cannot be recognized as either logic low or logic high, the entire process is known as metastability).



The above figure shows the restricted region (shaded region) for a flip-flop whose hold time is negative. The following diagram illustrates the restricted region of a D flip-flop. *D* is the input, *Q* is the output, and *clock* is the clock signal. If *D* changes in the restricted region, the flip-flop may not behave as expected, means *Q* is unpredictable.



To avoid setup time violations:

- The combinational logic between the flip-flops should be optimized to get minimum delay.
- Redesign the flip-flops to get lesser setup time.
- Tweak launch flip-flop to have better slew at the clock pin, this will make launch flip-flop to be fast there by helping fixing setup violations.
- Play with clock skew (useful skews).

To avoid hold time violations:

- By adding delays (using buffers).
- One can add lockup-latches (in cases where the hold time requirement is very huge, basically to avoid data slip).

* *may be expected one*: which means output is not sure, it may be the one you expect. You can also say "may not be expected one". "*may*" implies uncertainty. Thanks for the readers for their comments.

9 Comments

Labels: Important Concepts

Parallel vs Serial Data Transmission

Parallel and serial data transmission are most widely used data transfer techniques. Parallel transfer have been the preferred way for transfer data. But with serial data transmission we can achieve high speed and with some other advantages.

In parallel transmission n bits are transfered simultaneously, hence we have to process each bit separately and line up them in an order at the receiver. Hence we have to convert parallel to serial form. This is known as overhead in parallel transmission.

Signal skewing is the another problem with parallel data transmission. In the parallel communication, n bits leave at a time, but may not be received at the receiver at the same time, some may reach late than others. To overcome this problem, receiving end has to synchronize with the transmitter and must wait until all the bits are received. The greater the skew the greater the delay, if delay is increased that effects the speed.

Another problem associated with parallel transmission is crosstalk. When n wires lie parallel to each, the signal in some particular wire may get attenuated or disturbed due the induction, cross coupling etc. As a result error grows significantly, hence extra processing is necessary at the receiver.

Serial communication is full duplex where as parallel communication is half duplex. Which means that, in serial communication we can transmit and receive signal simultaneously, where as in parallel communication we can either transmit or receive the signal. Hence serial data transfer is superior to parallel data transfer.

Practically in computers we can achieve 150MBPS data transfer using serial transmission where as with parallel we can go up to 133MBPS only.

The advantage we get using parallel data transfer is reliability. Serial data transfer is less reliable than parallel data transfer.